COMSATS UNIVERSITY ISLAMABAD
Lahore Campus
Department of Computer Science

■ Lab Mid Examination

Course Title:	DevOps for Cloud Computing	Course Code:	CSC418	Credit Hours:	3(2,1)
Course Instructor/s:	Dr. M. Hasanain Ch	Program Name:	BCS		
Semester:	6th	Batch:	FA23		
Due Date:	**23 April 2026**	**Maximum Marks:**		**25**	

Q1. [CLO: 5; Bloom Taxonomy Level <Analyzing / Applying / Evaluating>]
Design, containerize, version, deploy, debug, and manage a cloud-native application using Azure, Docker, Kubernetes, and Git/GitHub. [Marks: 8 + 5 + 8 + 4 = 25]
You are required to design and implement a complete production style DevOps workflow for a cloud-hosted application. Unlike a basic deployment task, this exam will evaluate your ability to:

- Configure cloud service architecture on Azure,
- Containerize and optimize an application using Docker,
- Generate and manage Docker images,
- Deploy and expose the application using Kubernetes,
- Use Git and GitHub professionally for version control,
- Troubleshoot deployment/runtime issues,
- Document the full workflow with evidence.

Scenario:
You have been hired as a junior DevOps engineer. A development team has given you a small full-stack application that includes:

- A front-end,
- A back-end API,
- A database connection.

Your task is to take this application from a local machine to a cloud environment using DevOps best practices.

Mandatory Condition:
The application must be deployed in such a way that:

1. It runs successfully in a Docker container,
2. The Docker image is pushed to Docker Hub,
3. The code is version-controlled on GitHub,
4. The application is deployed to Kubernetes,
5. The final deployed application is accessible publicly.

Part 1: Docker Installation, Configuration, and Image Engineering **[8 Marks]**

Perform the following tasks on your chosen application:

1. Install and verify Docker on your system. **[1 Marks]**
   Show proof using appropriate verification commands.
2. Create a well-structured Dockerfile for the application. **[3 Marks]**
   The Dockerfile must:
   use a proper base image,
   define a working directory,
   copy only required files,
   install dependencies,
   expose the correct port,
   and start the application correctly.
3. Build a Docker image using a **YOUR REG NO.** as a tag name. **[1 Marks]**
4. Run the container locally and verify that the application is accessible in browser or through API testing. **[1 Marks]**
5. Optimize the image by applying at least two Docker best practices such as: **[2 Marks]**
   reducing image size,
   using. dockerignore,
   minimizing layers,
   or separating build/runtime concerns.
   Briefly justify what you improved.

Part 2: Git, GitHub, and Version Control Workflow **[5 Marks]**

You are required to manage the complete project using Git and GitHub.

1. Initialize a local Git repository and connect it with a GitHub repository. **[1 Mark]**
2. Push the full source code including Docker-related files to GitHub. **[1 Mark]**
3. Use Git commands properly to demonstrate: **[1 Mark]**
   - git add
   - git commit
   - git push
   - git pull
4. Create a second commit after making a small improvement or fix in the application, then push again. **[1 Mark]**
5. Maintain a meaningful commit history with proper commit messages. **[1 Mark]**

Part 3: Cloud Deployment with Azure and Kubernetes **[8 Marks]**

You must deploy the containerized application to the cloud using Azure and Kubernetes.

1. Create or configure the required Azure cloud resources for deployment.
   You may use Azure services as needed, but your design must clearly show the service architecture being used. **[2 Mark]**
2. Push your Docker image to Docker Hub and use that image in Kubernetes deployment. **[1 Mark]**
3. Create Kubernetes YAML files for: **[2 Mark]**
   - a **Deployment**
   - a **Service**
4. Deploy the application on Kubernetes and ensure the desired number of pods are created successfully. **[1 Mark]**
5. Expose the application to the public and provide a working IP. **[1 Mark]**

6. Demonstrate scaling by changing the number of replicas and showing the updated running pods. **[1 Mark]**

Part 3: Troubleshooting and DevOps Analysis **[4 Marks]**
After deployment, perform **any one realistic troubleshooting case** and document it clearly. Examples:

1. wrong container port mapping,
2. failed pod startup,
3. image pull error,
4. application crash due to missing environment variable,
5. service not accessible externally,
6. Git push rejected due to remote conflict.

You must:

1. identify the issue,
2. explain how you diagnosed it,
3. apply the fix,
4. and show the corrected result.

Required Submission

Submit a **PDF** containing all of the following:

1. File name must be your **Reg No**.
2. GitHub repository link
3. Docker Hub image link
4. Azure/Kubernetes public URL or public IP
5. screenshots of:
    - Docker installation verification
    - Docker image build
    - running container
    - Git commands and commit history
    - Docker Hub pushed image
    - Kubernetes deployment and service
    - running pods
    - public access of application
    - troubleshooting evidence before and after fix
6. all YAML files and Dockerfile
7. brief explanation of Azure service architecture used

Important Instructions

- Use your own semester project or a similar application with front-end, back-end, and database support.
- Copy-paste of another student's repository, Dockerfile, or YAML files will result in zero marks.
- Marks will be awarded for both correctness and professional DevOps practice.

- A working final deployment without proper Git/Docker/Kubernetes evidence will receive partial marks only.
- Viva will be conducted to verify your work on **April 23, 2026**.